

Secure Development: Coding Security

1. SQL Injection
2. Output Handling
-
3. Programming Languages Problem
4. Java Security
5. Java Security Demo

Lecture 6 - Review

Main points to take home

- **A big picture of digital cash and Bitcoin as a cryptocurrency**
 - Digital Cash, proposed by David Chaum
 - Bitcoin vs Traditional Banking System vs Digital Cash
- **Some cryptographic functions and Data structures used in Bitcoin**
 - Message Digest / Hash function
 - Public key cryptography with Digital Signature
 - Linked List, Binary Tree, and Merkle Tree

Main points to take home

- **A big picture of digital cash and Bitcoin as a cryptocurrency**
 - Digital Cash, proposed by David Chaum
 - Bitcoin vs Traditional Banking System vs Digital Cash
- **Some cryptographic functions and Data structures used in Bitcoin**
 - Message Digest / Hash function
 - Public key cryptography with Digital Signature
 - Linked List, Binary Tree, and Merkle Tree

Main points to take home

We have learned about the Bitcoin Protocol, which underpins the Blockchain technology

- Bitcoin Addresses
- Bitcoin Distributed Ledger: a chain of Blocks
- Transactions in Bitcoin and Blockchain
- Consensus Mechanism: Proof-of-Work

Main points to take home

We have also learned about Bitcoin Security and Privacy

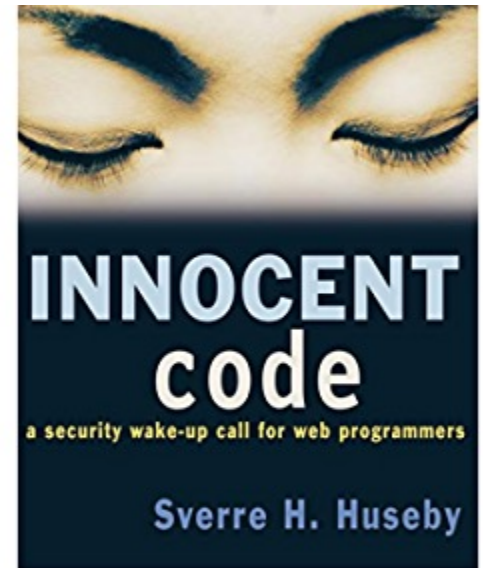
- Authentication → Public Key Crypto & Digital Signatures
 - Am I paying the right person? Not some other impersonator?
- Availability → Broadcast messages to the P2P network
 - Can I make a transaction anytime I want?
- Integrity → Digital Signatures & Cryptographic Hash
 - Is the coin double-spent?
 - Can an attacker reverse or change transactions?
- Confidentiality → Pseudonymity
 - Are my transactions private? Anonymous?

Lecture 7

Coding and Security

Outline

1. SQL Injection
2. Output Handling
-
3. Programming Languages Problem
4. Java Security
5. Java Security Demo

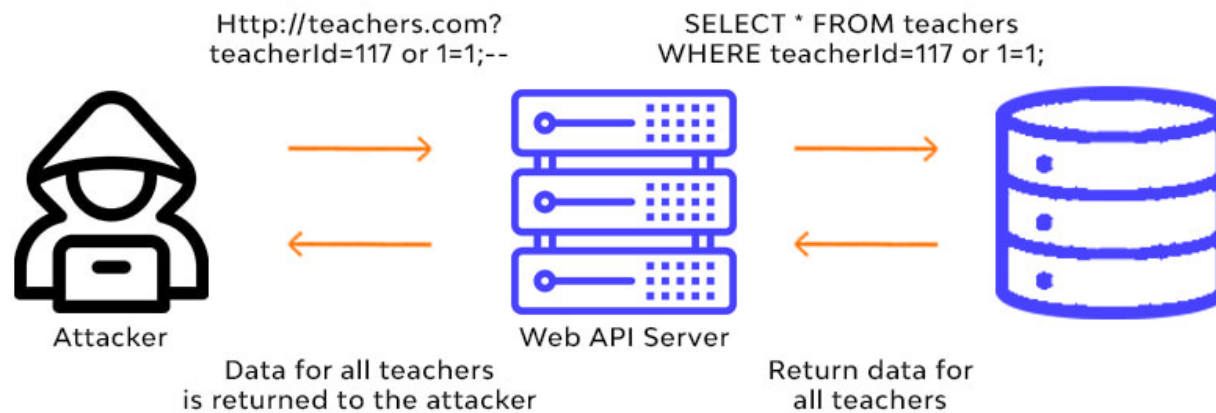


★★★★★ 5

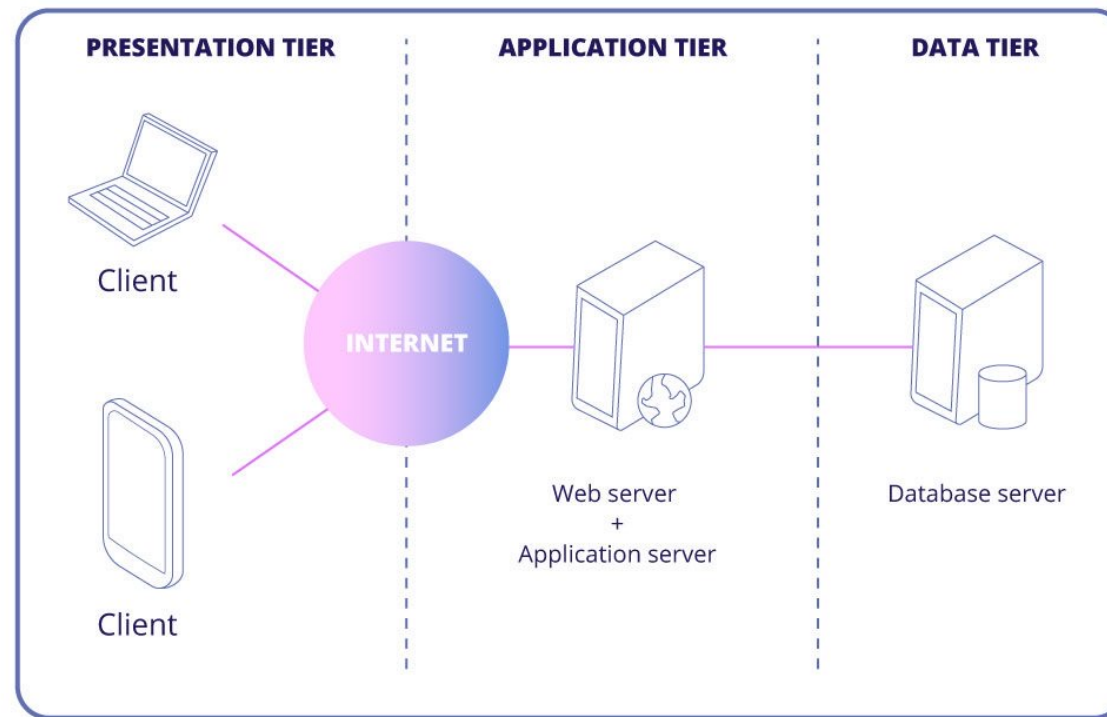
The book “Innocent Code” by Sverre Huseby has many useful examples.

SQL Injection

SQL Injection



Web Application Architecture



mobidev

<https://mobidev.biz/blog/web-application-architecture-types>

Passing Data to Subsystem

- Many applications pass data to subsystems.
 - SQL database.
 - Operating system.
 - Command interpreters.
- Often this is done by building strings and passing them to the subsystem.
 - The subsystem then parses the strings to extract meaning.
- Most of the content of the string is just plaintext, or data.
- Occasionally the string will contain characters with special meanings called meta-characters.
 - An attacker can pass meta-characters as plaintext.

SQL Injection: Ideas

- Many systems pass SQL queries to a database subsystem.
- The queries are often constructed from **user input**.
- These queries will contain **meta-characters** which can be misused by an attacker.
 - For example, **'** will switch from command mode to input mode and vice versa.
 - A user can put **'** in their input, getting SQL to switch to command mode.
 - They have injected a command into the SQL system.

SQL with Meta-characters: Example

- We have a SQL table called `User` with a column called `RealName`.
- We have a program that gets the name from the user and stores it in the Java String variable `name`.

- We construct a query string in Java by:

```
String query = "INSERT INTO User (RealName) VALUES ('" + name + "')";
```

- If `name` contains Nguyen Truong then the query becomes:

```
INSERT INTO User (RealName) VALUES ('Nguyen Truong')
```

- And everything works fine.

SQL with Meta-characters: Example

- If John O'Donnell entered his name into the system, the query would become:

```
INSERT INTO User (RealName) VALUES ('John O'Donnell')
```

- This will not work because SQL thinks the name is John O and the text after the second ' causes an error.

SQL Injection Attack – Comment out

- We are using a database called **User** that stores usernames and passwords in columns called **UserName** and **Password**.
- A login page will get the relevant information from the user and store in two variables **user** and **pass**.
- We then construct a query string in Java to access the database.

```
String Query = "SELECT * FROM User "  
+ "WHERE UserName='" + user + "' "  
+ "AND Password='" + pass + "'";
```

- This only returns a row if the user entered **username** and **password** are correct.
 - **Warning: this is not a good way to check passwords.**

SQL Injection Attack – Comment out

- If the input text was “Nguyen” and “abcdefgh” then the query submitted to the database would be:

```
SELECT * FROM User WHERE UserName='Nguyen' AND  
Password='abcdefgh'
```

- This would work as expected.
- On the other hand, if the input for name was “Nguyen' --” and the password was empty (or any), the generated SQL would be:

```
SELECT * FROM User WHERE UserName='Nguyen' --' AND  
Password=''
```

- The -- introduces a **SQL comment**, which invalidates the password check, provided there is a user called Nguyen.
- The user gets in without knowing the password!

SQL Injection Attack – Comment out

Database	Single-Line	Multi-Line
Oracle	--	/* */
SQL Server	--	/* */
MySQL	-- OR #	/* */
PostgreSQL	--	/* */

SQL Injection Attack Demo

<https://www.codingame.com/playgrounds/154/sql-injection-demo/sql-injection>

SQL Injection Attack – Boolean Precedence

- Some databases such as Access do not recognise SQL comments. In this case we can use the **Precedence of Boolean connectives** to bypass the password check.
- Enter user **Nguyen' OR 'a'='b** and leave password blank (or any). This becomes:

```
SELECT * FROM User WHERE UserName='Nguyen' OR 'a'='b'  
AND Password=''
```

- AND has a higher precedence than OR, and both parts of the AND are false, leading to:

```
SELECT * FROM User WHERE UserName='Nguyen' OR FALSE
```

- But only one part of the OR needs to be true!

SQL Injection Attack Demo

<https://www.codingame.com/playgrounds/154/sql-injection-demo/sql-injection>

How to store Users' passwords

```
1  export const seedUsers = [{
2    username: 'admin',
3    type: 'ADMIN',
4    firstname: 'Fadmin',
5    lastname: 'Ladmin',
6    password: '$2b$04$j0aSK.Dyq7Q9N.r6d0uIa0Gr0e7sI4rGUn0JNcaXcPCv.490tjwpi',
7    otpSecret: 'H6BNKK027DPLCATGEJAZNWQV4LW0TMRA',
8    organisation: 'organisation_system',
9    email: 'admin@example.com',
10   resetPasswordRequests: [],
11   description: 'I am an admin user.',
12   emailNotificationsActivated: true,
13   emailNotificationsStatus: { expiringNotification: false },
14   deleted: null,
15   id: 'replaced_at_runtime2',
16   createdAt: 1591134065000,
17   expiredAt: 1991134065000
18 },
19 {
```

How to check Users' passwords

```
/* randomly generate a secret for Time-based One Time Password*/
```

```
const otpSecret = mfa.generateSecret();
```

```
/* all ok; change the user's password */
```

```
const hashedPw = await bcrypt.hash(newPassword, config.bcrypt.saltround);
```

```
const updateResult = await db.collections!.users_collection.findOneAndUpdate(  
  {
```

```
login: async (parent: Record<string, unknown>, args: any, context: any): Promise<Record<string, unknown>> => {  
  const { req }: { req: Express.Request } = context;  
  const result = await db.collections!.users_collection.findOne({ deleted: null, username: args.username });  
  if (!result) {  
    throw new GraphQLError('User does not exist.', { extensions: { code: ApolloServerErrorCode.BAD_USER_INPUT } });  
  }
```

```
const passwordMatched = await bcrypt.compare(args.password, result.password);
```

```
if (!passwordMatched) {  
  throw new GraphQLError('Incorrect password.', { extensions: { code: ApolloServerErrorCode.BAD_USER_INPUT } });  
}
```

SQL Injection: Batched SQL Statements with Semicolon

- This query looks up a customer in a table based on an integer customer ID, stored in a string `cust`.

```
String Query = "SELECT * FROM Customer WHERE  
CustID=" + cust
```

- There is no `'`, but consider the following `cust` string:

```
1; DELETE FROM Customer
```

- The `;` character is used to run SQL commands in *batch mode*, one after the other.

SQL Injection: Batched SQL Statements with Semicolon

- This is faithfully translated as the string:

```
SELECT * FROM Customer WHERE CustomerID=1;  
DELETE FROM Customer
```

- SQL will expect an integer for `CustomerID` and will recognise the `;` following the 1 as terminating the input, going back to command mode and deleting customer 1.
- Not all databases support batch mode.

SQL Injection with UNION SELECT

- Now let us try and get information out of a database.
- We cannot just inject a `SELECT` statement in our input data, because that would return more than one result sets.
- `UNION SELECT` will combine the results sets so that we can mix sensitive data with genuine data that we are allowed to access.
- In this example we have a second table called `Customers` contains columns: `CustomerID`, `CustomerName`, `ContactName` and `Address`.
- A query for accessing a customer according to a numeric category would be:

```
String Query = "SELECT CustomerName, ContactName,  
Address FROM Customers WHERE CustomerID=" + custID;
```

- Obtain all items related to a customer in a given ID.

SQL Injection with UNION SELECT

- Now let `custID` contain:

```
1 UNION SELECT 'a',UserName,Password from Users
```

- The query as seen by SQL is:

```
SELECT CustomerName, ContactName, Address FROM  
Customers WHERE CustomerID=1 UNION SELECT  
'a',UserName,Password from Users
```

- We get the items from `customerID = 1` together with all the usernames and passwords.
- The `'a'` is there to make sure that the second SELECT produces a results set with the same form as the first: (string, string, string).

Try out: https://www.w3schools.com/sql/trysql.asp?filename=trysql_view1

SQL Injection: Sub-Queries

- We can inject complete SQL commands as sub-queries, potentially doing a lot of damage.
- This example runs a SQL statement to update a user's address based on the user input.
- It is in a Java program.
 - Address is a Java String.
 - `userId` is an Integer.

```
Query="UPDATE Users SET Address='"+address+"' "  
+ "WHERE UserID="+userId
```

SQL Injection: Sub-Queries

- We enter the following address:

```
' || (SELECT Password FROM Users  
WHERE username='Nguyen') || '
```

- || concatenates strings.

- If the attacker's user ID is 1234 then the query becomes:

```
UPDATE Users SET Address=' ' || (SELECT Password  
FROM Users WHERE username='Nguyen') || '  
WHERE UserID=1234
```

- The attacker's address will now contain Nguyen's password!
- We can make the same attack using SQL Injection with Batched SQL Statements using ; in systems that support batch operations.

SQL Injection: URL parameters

- Here is a real-life example of injecting code into a SQL query.
- A payment system web page could be accessed with the URL:
 - `http://pay.example/default.asp?id=3;SHUTDOWN`
- The string `3;SHUTDOWN` was injected into the SQL as the `id` number and duly shut down the database, several times.
- This suggestion was made in a discussion group, and the person making the suggestion, together with the unbelievers who tried it out; and it worked. Not long after, they were visited by the police.
- Note that I am not making a suggestion, merely reporting something that is public knowledge!

SQL Injection: Obtaining Info.

Helpful Error Messages

- The previous attacks have relied on knowing the names of the tables and columns.
- This can often be obtained from **helpful error messages**.
 - Discovered by David Litchfield in his paper:
 - Web Application Disassembly with ODBC Error Messages (<http://www.davidlitchfield.com/WebApplicationDisassemblywithODBCErrorMessages.pdf>)
 - An attempt to access an invalid column can result in an error message that lists all the columns in the table being attacked.
 - Alternatively, columns can be obtained one by one.

Error Messages: An Example

- The following parameters were appended to a URL:

`?id=1+OR+qwe=1`

- `+` is the URL code for a *space character*.
- So, the id parameter is: `id=1 OR qwe=1`
 - There is no column called `qwe`.
 - The underlying system used ODBC (a C version of JDBC) to access the database, and ODBC provided a helpful error message that listed all the columns in the table being accessed.
- This was done in the ODBC library and not the programmer's code.
- Making it hard to avoid.
- The moral here is to avoid helpful error messages for users that could also be seen by an attacker.

SQL Injection: Obtaining Info.

Using GROUP to get Columns

- Assume the underlying application constructs the query:
`"SELECT * FROM News WHERE id=" + id`
- Attacker enters `id` as `1 HAVING 1=1` for the `id` to produce:
`SELECT * FROM News WHERE id=1 HAVING 1=1`
- The error message is: “Attribute news.id must be GROUPed or used in an aggregate function”.
- This tells the attacker the table is called `News` and that the first column is `id`.
- Try again with `1 GROUP BY id HAVING 1=1`
- The error message then says that the next column is `Title`.
- Repeat until there are no error message to get all the columns.

SQL Injection



Solution

- Input Validation or Sanitization
- Parametrised queries
- Stored Procedures

Unchecked Meta-character

- The problem is not the `--` comment in SQL nor the Boolean Precedence.
- The problem is in the Java program which has allowed the metacharacter `'` to be entered as data.
 - When SQL parses its query, it used the first `'` to switch from *command mode* to *data mode*.
 - The second `'` switches back to *command mode*, and so on.
 - This has allowed us to enter commands as data.
- This is a tricky problem because `'` can be a valid part of a name.
 - O'Donnell, for example.

Input Validation

- Validation shouldn't only be applied to fields that allow users to type in input
 - You should also take care of the following situations in equal measure:
- Use *regular expressions* as whitelists for structured data (such as name, age, income, survey response, zip code) to ensure strong input validation.
- In case of a fixed set of values (such as drop-down list, radio button), determine which value is returned.
 - The input data should match one of the offered options exactly.

Input Validation

- The validation process is aimed at verifying whether or not the type of input submitted by a user is allowed.
- Input validation makes sure it is the accepted type, length, format, and so on
- Different Approaches
 - Escaping meta-characters
 - Reject bad inputs
 - Accept only good inputs

Parametrised Query

- Parametrised queries are a means of pre-compiling an SQL statement so that you can then supply the parameters in order for the statement to be executed.
- This method makes it possible for the database to recognise the code and distinguish it from input data.
- The user input is automatically quoted and the supplied input will not cause the change of the intent
 - Thus, this coding style helps mitigate an SQL injection attack.

```

1 String query = "SELECT userName, balance FROM accounts"
2   + "WHERE userID=" + request.getParameter("userID") +
3   "and password='" + request.getParameter("Password") + "'";
4
5 try
6 {
7     Statement statement = connection.createStatement();
8     ResultSet rs = statement.executeQuery(query);
9     while (rs.next())
10    {
11        page.addRow(rs.getString("userName"),
12                    rs.getFloat("balance"));
13    }
14 }
15 catch (SQLException e)
16 {}
17

```

```

1 String query = "SELECT userName, balance "+
2   "FROM accounts WHERE userID = ?
3   and password = ?";
4
5 try {
6     PreparedStatement statement = connection.prepareStatement(query);
7     statement.setInt(1, request.getParameter("userID"));
8     ResultSet rs = statement.executeQuery();
9     while (rs.next())
10    {
11        page.addRow(rs.getString("userName"),
12                    rs.getFloat("balance"));
13    }
14 } catch (SQLException e)
15     { ... }
16

```

Parametrised Query: Example

Stored Procedures

- Similar to Parametrised Queries
 - Input values are passed as parameters
 - Inputs are as string literals in SQL and NOT as a part of the SQL query
- Differences:
 - Stored Procedures are executed at DBMS, stored and associated to a schema in DBMS
 - Each query is processed more rapidly than a straight query, because the server pre-compiles them.
 - Each query need only be written once. It can be executed as many times as needed, even across different sessions and different connections.
 - Allows queries to include programming constructs (such as loops, conditional statements, and error handling) that are either impossible or difficult to write in SQL alone.

SQL Injection Summary

- Concept of passing data to sub-system
- Different types of SQL Injection attacks
 - Comment out
 - Boolean Precedence
 - Batched SQL statements using Semicolon
 - Union Select
 - Sub-Queries
 - URL parameters
- Obtain Information for SQL injection attacks
 - Helpful Error Messages
- Prevention Methods
 - Input validation & sanitization
 - Parametrised Query
 - Stored Procedures

Quiz

1. Give two examples of a SQL injection attack and state what coding standards are necessary to defeat this attack.
2. Show how UNION SELECT and Sub Queries can be used to run an attacker's SELECT statement on a target machine.
3. How can details of the database tables on the target machine be found?